ICPC Team Reference Notebook

C++ first, Python where it helps. Every C++ snippet compiles (C++17). Use Ctrl+F or the contents list to jump.

Contents

1. Setup and Template....3
1.1 C++ starter template....3
1.2 Compile, run, stress test....3
1.3 Python starter template....3
1.4 Complexity guide....3
2. C++ Syntax Essentials....3
2.1 Types, limits, overflow....3
2.2 Input and output....3
2.3 Lambdas, auto, structured bindings....3
2.4 Characters, strings and numbers....3
2.5 Structs, arrays, comparators....3
2.6 Random, timing....4
2.7 Input patterns in C++....4
2.8 Output patterns in C++....4
2.9 Functions, references, pointers....4
2.10 Control flow and operators....4
2.11 Templates, structs, classes, overloading....4
2.12 String member functions (C++)....5
2.13 algorithm and numeric catalog (C++)....5
3. STL Containers....5
3.0 Container cheat sheet....5
3.1 vector....5
3.2 pair, tuple, array....5
3.3 stack, queue, deque....6
3.4 priority_queue....6
3.5 set and multiset....6
3.6 map and unordered_map....6
3.7 bitset and iterators....6
4. STL Algorithms....6
4.1 Sorting and comparators....6
4.2 Binary search: lower_bound, upper_bound....6
4.3 Sum, accumulate, prefix sums, fill....6
4.4 Find, count, min, max....6
4.5 Remove, erase, unique, replace....7
4.6 Permutations, subsets, combinations....7
4.7 Set operations and merging (sorted input)....7
4.8 GCD, LCM, min, max, math helpers....7
4.9 Bit builtins....7
5. Strings....7
5.1 String operations....7
5.2 Prefix function (KMP) and Z-function....7
5.3 Rolling hash (mod 2^61 - 1)....8
5.4 Trie....8
6. Python Quick Reference....8
6.1 C++ and Python side by side....8
6.2 Input, output, lists....8
6.3 Accumulate, find, count, remove....8
6.4 dict, set, Counter, deque....9
6.5 heapq and bisect....9
6.6 Strings....9
6.7 math, itertools, functools, bits....9
6.8 Graph code in Python....9
6.9 Python speed tips....9
6.10 Python input patterns (every format)....9
6.11 Python input: tokens, EOF, huge input....9
6.12 Python syntax essentials....10
6.13 Functions, lambdas, recursion....10
6.14 Classes, dataclasses, sorting objects....10
6.15 Comprehensions, slicing, unpacking....11
6.16 Python method catalog....11
7. Graphs....11
7.1 Building a graph....11
7.2 BFS: distances, grid, multi-source....12
7.3 DFS, components, cycles, topological sort....12
7.4 Shortest paths....12
7.5 DSU and minimum spanning tree....13
7.6 Bridges, articulation points, SCC....13
7.7 Trees: subtree size, diameter, LCA....13
7.8 Max flow (Dinic) and bipartite matching....13
7.9 Bipartite check....14
8. Dynamic Programming....14
8.1 Max subarray sum and max product subarray....14
8.2 Longest increasing subsequence....14
8.3 LCS and edit distance....14
8.4 Knapsack and coin change....15
8.5 Bitmask DP (travelling salesman)....15
8.6 Subset sums (SOS DP) and submask enumeration....15
8.7 Digit DP skeleton....15
8.8 Prefix sums and difference arrays....15
9. Data Structures....15
9.1 Fenwick tree (BIT)....15
9.2 Segment tree (iterative, point update)....15
9.3 Lazy segment tree (range add, range sum)....15
9.4 Sparse table (static range minimum)....16
9.5 Monotonic stack and deque....16
9.6 Two pointers and coordinate compression....16
9.7 Ordered set (policy based data structure)....16
10. Number Theory and Math....16
10.1 Modular arithmetic, fast power, nCr....16
10.2 GCD, extended GCD, modular inverse (any modulus)....17
10.3 Sieve of Eratosthenes and smallest prime factor....17
10.4 Factorization, divisors, Euler phi....17
10.5 Matrix exponentiation....17
10.6 Miller-Rabin primality (64-bit)....17
10.7 Formulas....17
11. Searching Tricks....18
11.1 Binary search on the answer....18
11.2 Ternary search (unimodal function)....18
12. Geometry....18
12.1 Points, cross product, orientation....18
12.2 Convex hull and polygon area....18
13. Bit Manipulation....18
13.1 Common tricks....18
13.2 Enumerating masks....18
14. Pitfalls and Constants....18
14.1 Contest checklist....18
14.2 Constants and tables....19

Quick lookup

I need	Section
Template, fast input/output	1.1, 2.2
C++ input formats (grid, EOF, queries)	2.7, 2.8
C++ functions, references, templates	2.9 to 2.11
C++ string / algorithm function list	2.12, 2.13
Python input formats (all kinds)	6.10, 6.11
Python syntax, functions, classes	6.12 to 6.14
Python slicing, comprehensions, methods	6.15, 6.16
What complexity fits n	1.4
Types, overflow, limits	2.1
Sum / accumulate / prefix sums	4.3, 8.8, 6.3
Find, count, max/min and index	4.4, 6.3
Remove / erase / unique	4.5, 6.3
Sort, custom comparator	4.1, 6.2
lower_bound / upper_bound	4.2, 6.5
Permutations, subsets, combinations	4.6
vector, set, map, priority_queue	3.1 to 3.6
Strings: split, find, KMP, hash, trie	5.1 to 5.4
C++ vs Python equivalents	6.1
Build a graph (adjacency list)	7.1
BFS, grid BFS, DFS	7.2, 7.3
Cycle detection, topological sort	7.3
Dijkstra, Floyd, Bellman-Ford, 0-1 BFS	7.4
DSU, Kruskal, Prim	7.5
Bridges, articulation points, SCC	7.6
LCA, diameter, subtree size	7.7
Max flow, bipartite matching	7.8, 7.9
Kadane, LIS, LCS, edit distance	8.1 to 8.3
Knapsack, coin change, bitmask DP	8.4 to 8.6
Fenwick, segment tree, sparse table	9.1 to 9.4
Monotonic stack, two pointers, compression	9.5, 9.6

I need	Section
gcd, modpow, inverse, nCr	10.1, 10.2
Sieve, factorize, phi, Miller-Rabin	10.3 to 10.6
Math formulas, constants	10.7, 14.2
Binary search on answer, ternary	11.1, 11.2
Geometry, convex hull	12
Bit tricks	4.9, 13
Common mistakes checklist	14.1

1. Setup and Template

1.1 C++ starter template

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;
using ull = unsigned long long;
using pii = pair<int, int>;
using vi = vector<int>;
const ll INF = 1e18;
const int MOD = 1e9 + 7;   // or 998244353
#define all(x) (x).begin(), (x).end()
#define sz(x) (int)(x).size()

void solve() {
    int n; cin >> n;
    vi v(n);
    for (auto& x : v) cin >> x;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int t = 1;
    cin >> t;            // remove if single test
    while (t--) solve();
}
```

All C++ snippets in this notebook assume this template (macros `all`, `sz`, types `ll`, `pii`, `vi`, constants `INF`, `MOD`) and C++17. Graph snippets assume 1-indexed nodes and global `n`.

1.2 Compile, run, stress test

```
g++ -std=c++17 -O2 -Wall -Wextra -Wshadow -o a a.cpp
g++ -std=c++17 -g -fsanitize=address,undefined -o a a.cpp
./a < in.txt > out.txt
```

```
# stress test: gen.py prints a random input, brute is slow
for i in $(seq 1 500); do
  python3 gen.py $i > in.txt
  ./a < in.txt > o1.txt
  ./brute < in.txt > o2.txt
  cmp -s o1.txt o2.txt || { echo DIFF $i; break; }
done
```

1.3 Python starter template

```
import sys
input = sys.stdin.readline
# sys.setrecursionlimit(10**6)   # only for recursive DFS

def solve():
    n = int(input())
    a = list(map(int, input().split()))
    print(sum(a))

def main():
    t = int(input())             # remove if single test
    for _ in range(t):
        solve()

main()
```

1.4 Complexity guide

n (max)	Complexity that fits
up to 10	O(n!) brute force, permutations
up to 20	O(2^n * n) bitmask DP
up to 500	O(n^3), Floyd Warshall
up to 5000	O(n^2)
up to 1e5	O(n log n): sort, BIT, segment tree
up to 1e6	O(n) or O(n log n) with small constant
up to 1e12	O(sqrt n)
up to 1e18	O(log n), binary search, fast power

Rule of thumb: C++ does about 1e8 simple operations per second. Python does about 1e7 (use PyPy, avoid deep recursion).

2. C++ Syntax Essentials

2.1 Types, limits, overflow

Type	Size	Range
`int`	32 bit	about +-2.1e9 (INT_MAX = 2147483647)
`unsigned`	32 bit	0 to 4.29e9
`long long`	64 bit	about +-9.2e18 (LLONG_MAX)
`unsigned long long`	64 bit	0 to 1.8e19
`__int128`	128 bit	about +-1.7e38 (no cin/cout)
`double`	64 bit	about 15 digits
`long double`	80 bit	about 18 digits
`char`	8 bit	-128 to 127

```
int a1 = 100000, b1 = 100000;
ll bad = a1 * b1;            // overflow: int * int
ll good = 1LL * a1 * b1;     // cast BEFORE multiplying
ll big = LLONG_MAX, small = LLONG_MIN;
int mx = INT_MAX, mn = INT_MIN;
__int128 z = (__int128)a1 * b1;
double d = 1.0 * a1 / b1;    // avoid integer division
int ceilDiv = (a1 + b1 - 1) / b1;   // ceil for positives
```

2.2 Input and output

```
// whitespace separated tokens
cin >> n >> s;
getline(cin, s);                 // whole line
// after cin >> n, call cin.ignore() before getline
while (cin >> x) { }             // read until EOF
cout << n << " " << s << "\n";   // "\n" is faster than endl
cout << fixed << setprecision(9) << 3.14159265 << "\n";
cout << setw(5) << setfill('0') << n << "\n";   // 00005
printf("%d %lld %.6f %s\n", n, p, 1.5, s.c_str());
// do not mix with cin if synced off
scanf("%d", &n);
cerr << "debug " << n << "\n";   // stderr, ignored by judge
// interactive problems: flush with endl or cout.flush()
cout << (a < b ? "YES" : "NO") << "\n";
```

2.3 Lambdas, auto, structured bindings

```
auto add = [&](int a, int b) { return a + b; };
// recursive lambda
function<int(int)> fib = [&](int i) -> int {
    return i < 2 ? i : fib(i - 1) + fib(i - 2);
};
pair<int, ll> pr = {1, 2};
auto [first, second] = pr;               // C++17
for (auto& [key, val] : mp) { val++; }   // map iteration
for (auto& e : v) e *= 2;                // modify in place
for (int e : v) { }                      // copy
auto it = v.begin();                     // iterator type
```

2.4 Characters, strings and numbers

```
int a1 = stoi("123");  ll b1 = stoll("123");
double d1 = stod("1.5");
string s1 = to_string(123);              // int to string
int digit = ch - '0';                    // '7' -> 7
char c2 = '0' + digit;                   // 7 -> '7'
// 'a'..'z' -> 0..25
int idx = ch - 'a';
isdigit(ch); isalpha(ch); isupper(ch); islower(ch);
toupper(ch); tolower(ch);
int bin = stoi("101", nullptr, 2);       // from base 2
string bits = bitset<32>(x).to_string(); // to binary string
string r = string(5, 'a');               // "aaaaa"
```

2.5 Structs, arrays, comparators

```
struct Item {
    int a, b;
    bool operator<(const Item& o) const {
        return a != o.a ? a < o.a : b > o.b;
    }
};
// set<Item>, sort(), priority_queue<Item> (pops LARGEST)
```

```
int arr[100005];                 // global arrays start at 0
memset(arr, 0, sizeof arr);      // only for 0 or -1
fill(arr, arr + 100005, 7);
int mat[100][100];
vector<vector<int>> grid(n, vector<int>(m, -1));
const int dx[] = {1, -1, 0, 0}, dy[] = {0, 0, 1, -1};
```

2.6 Random, timing

```
mt19937 rng(chrono::steady_clock::now()
                .time_since_epoch().count());
int r1 = rng() % 100;
int r2 = uniform_int_distribution<int>(1, 6)(rng);
shuffle(all(v), rng);
auto t0 = clock();
double secs = (clock() - t0) / (double)CLOCKS_PER_SEC;
```

2.7 Input patterns in C++

```
// 1) T test cases, each with n and an array
int T; cin >> T;
while (T--) {
    int N; cin >> N;
    vector<ll> arr(N);
    for (auto& e : arr) cin >> e;
}
// 2) n x m grid of characters
int R, C; cin >> R >> C;
vector<string> grid(R);
for (auto& row : grid) cin >> row;      // grid[i][j]
// 3) n x m matrix of integers
vector<vector<int>> mat(R, vector<int>(C));
for (auto& row : mat) for (auto& e : row) cin >> e;
// 4) m edges "u v" (1-indexed)
int E; cin >> E;
for (int i2 = 0; i2 < E; i2++) {
    int u, v2; cin >> u >> v2;
}
// 5) queries with a type: "1 x y" or "2 x"
int Q; cin >> Q;
while (Q--) {
    int type; cin >> type;
    if (type == 1) { int xx, yy; cin >> xx >> yy; }
    else { int xx; cin >> xx; }
}
// 6) a whole line with spaces after reading a number
int num; cin >> num;
cin.ignore(numeric_limits<streamsize>::max(), '\n');
string line; getline(cin, line);
// 7) unknown count: read until end of file
vector<int> all_nums; int val;
while (cin >> val) all_nums.push_back(val);
// 8) all lines until EOF
vector<string> lines;
while (getline(cin, line)) lines.push_back(line);
// 9) numbers in one line, count unknown
getline(cin, line);
stringstream ss2(line); vector<int> nums;
while (ss2 >> val) nums.push_back(val);
// 10) comma separated "3,4,5"
string csv = "3,4,5"; stringstream sc(csv); string tok;
while (getline(sc, tok, ',')) nums.push_back(stoi(tok));
// 11) digits of a number given as a string
string digs; cin >> digs;
for (char d2 : digs) { int dv = d2 - '0'; }
// 12) read a double, a char, a long long
double dd; char cc; ll big; cin >> dd >> cc >> big;
```

2.8 Output patterns in C++

```
vector<int> res = {1, 2, 3};
vector<string> grid = {"ab", "cd"};
int T = 2;
// space separated on one line
for (int i2 = 0; i2 < sz(res); i2++)
    cout << res[i2] << " \n"[i2 == sz(res) - 1];
// one per line
for (int e : res) cout << e << "\n";
// grid
for (auto& row : grid) cout << row << "\n";
// "Case #i: ans" (Google / ICPC style)
for (int tc = 1; tc <= T; tc++)
    cout << "Case #" << tc << ": " << 42 << "\n";
// fixed decimals
cout << fixed << setprecision(6) << 2.0 / 3 << "\n";
// YES / NO
bool okk = true;
cout << (okk ? "YES" : "NO") << "\n";
// -1 if impossible
ll ansv = INF;
cout << (ansv == INF ? -1 : ansv) << "\n";
// build big output in one string, print once
string outs;
for (int e : res) outs += to_string(e) + "\n";
cout << outs;
```

2.9 Functions, references, pointers

```
// pass by value (copy), by reference (no copy, can modify)
// and by const reference (no copy, read only)
int byValue(int x) { return x * 2; }
void byRef(int& x) { x++; }
ll sumOf(const vector<int>& v) {
    ll s = 0;
    for (int x : v) s += x;
    return s;
}
// pass vectors by & or const&, never by value in loops
void fillVec(vector<int>& v, int n) { v.assign(n, 7); }
// default arguments
ll powr(ll a, ll b = 2) {
    ll r = 1;
    while (b--) r *= a;
    return r;
}
// overloading: same name, different parameters
int mx(int a, int b) { return a > b ? a : b; }
ll mx(ll a, ll b) { return a > b ? a : b; }
// return several values
pair<int, int> minMax(const vector<int>& v) {
    return {*min_element(v.begin(), v.end()),
            *max_element(v.begin(), v.end())};
}
tuple<int, int, int> three() { return {1, 2, 3}; }
// use: auto [lo, hi] = minMax(v);
// recursion with memo
map<ll, ll> memoF;
ll fibm(ll k) {
    if (k < 2) return k;
    if (memoF.count(k)) return memoF[k];
    return memoF[k] = fibm(k - 1) + fibm(k - 2);
}
// pointers (rarely needed)
void pointers() {
    int x = 5;
    int* p = &x;      // address of x
    *p = 6;           // x is now 6
    int& r = x;       // reference: another name for x
    r = 7;            // x is now 7
}
```

2.10 Control flow and operators

```
for (int i2 = 0; i2 < n; i2++) { }        // 0..n-1
for (int i2 = n - 1; i2 >= 0; i2--) { }   // reverse
for (int i2 = 0; i2 < n; i2 += 2) { }     // step 2
for (int i2 = 1; i2 * i2 <= n; i2++) { }  // up to sqrt(n)
while (n > 0) { n /= 10; }                // digits loop
do { } while (false);
if (a > b) { } else if (a == b) { } else { }
switch (a) {
    case 1: break;
    default: break;
}
// break exits the loop, continue skips to next iteration
// leave nested loops: use a flag or a lambda with return
bool found = false;
for (int i2 = 0; i2 < n && !found; i2++)
    for (int j2 = 0; j2 < m; j2++)
        if (i2 + j2 == 3) { found = true; break; }
int r2 = 7 / 2;      // 3 (integer division)
int r3 = -7 / 2;     // -3 (rounds toward zero)
int r4 = -7 % 3;     // -1 (sign follows left side)
int r5 = 7 % 3;      // 1
a += 2; a -= 2; a *= 2; a /= 2; a %= 2;
a++; ++a; a--; --a;
int t2 = (a > b) ? a : b;                 // ternary
bool both = (a > 0 && b > 0), either = (a > 0 || b > 0);
int bw = a & b, bo = a | b, bx = a ^ b, bn = ~a;
int sl = a << 2, sr = a >> 1;
```

2.11 Templates, structs, classes, overloading

```
// generic function
template <typename T>
T myMax(T a, T b) { return a > b ? a : b; }
template <typename T>
void printVec(const vector<T>& v) {
    for (auto& x : v) cout << x << " ";
    cout << "\n";
}
// struct with constructor, method, comparison
struct Edge2 {
    int u, v;
    ll w;
```

```
    Edge2(int u, int v, ll w) : u(u), v(v), w(w) {}
    bool operator<(const Edge2& o) const { return w < o.w; }
};
// usage: vector<Edge2> es; es.emplace_back(1, 2, 5);
// sort(es.begin(), es.end());   uses operator<
// class with private data
class Counter {
    int cnt = 0;
  public:
    void inc() { cnt++; }
    int get() const { return cnt; }
};
// print a struct with cout
struct Pt { int x, y; };
ostream& operator<<(ostream& os, const Pt& p) {
    return os << "(" << p.x << ", " << p.y << ")";
}
// hash a pair for unordered_map / unordered_set
struct PairHash {
    size_t operator()(const pair<int, int>& p) const {
        return hash<ll>()(((ll)p.first << 32) ^ p.second);
    }
};
// unordered_map<pair<int,int>, int, PairHash> cellMap;
// map<pair<int,int>, int> works without a hash
// debug print helper
template <typename... A>
void dbg(A... a) {
    ((cerr << a << " "), ...);
    cerr << "\n";
}
```

2.12 String member functions (C++)

Call	Does
`s.size()`, `s.length()`	length
`s.empty()`	true if ""
`s[i]`, `s.at(i)`, `s.back()`	character access
`s.substr(pos, len)`	copy of part (len optional)
`s.find(t)`, `s.find(t, from)`	first index or `string::npos`
`s.rfind(t)`	last index or `npos`
`s.find_first_of("aeiou")`	first index of any of those chars
`s.find_first_not_of(' ')`	first non-space (for trimming)
`s.append(t)`, `s += t`, `s + t`	concatenate
`s.push_back(c)`, `s.pop_back()`	add or remove last char
`s.insert(pos, t)`	insert text at pos
`s.erase(pos, len)`	delete part
`s.replace(pos, len, t)`	overwrite part
`s.compare(t)`	<0, 0, >0 like strcmp
`s.c_str()`	C string for printf
`string(n, c)`	c repeated n times
`to_string(x)`	number to string
`stoi`, `stol`, `stoll`, `stod`	string to number
`reverse(all(s))`, `sort(all(s))`	reverse, sort letters
`count(all(s), 'a')`	count a char
`s.starts_with(t)` (C++20)	else `s.rfind(t, 0) == 0`
`tolower(c)`, `toupper(c)`	single char case
`transform(all(s), s.begin(), ::tolower)`	whole string lower

2.13 algorithm and numeric catalog (C++)

Function	Result
`sort`, `stable_sort`	sort range
`reverse`, `rotate`	reorder
`min_element`, `max_element`	iterator to min/max
`minmax_element`	pair of iterators
`accumulate(b, e, init)`	sum (or fold with op)
`partial_sum`, `adjacent_difference`	prefix sums, diffs
`iota(b, e, start)`	fill start, start+1, ...
`fill(b, e, x)`, `fill_n(b, n, x)`	set all to x
`count`, `count_if`	how many match
`find`, `find_if`, `find_if_not`	iterator to first match
`search(b, e, b2, e2)`	first occurrence of a subrange
`any_of`, `all_of`, `none_of`	bool tests
`remove`, `remove_if`	move kept items front (then erase)
`unique`	collapse adjacent duplicates
`replace`, `replace_if`	substitute values
`copy`, `copy_if`	copy into another range
`transform(b, e, out, f)`	apply f to each element
`lower_bound`, `upper_bound`	binary search iterators
`binary_search`, `equal_range`	bool, pair of bounds
`next_permutation`, `prev_permutation`	step permutations
`nth_element`, `partial_sort`	k-th element, top k
`merge`, `set_union`, `set_intersection`	sorted range ops
`swap`, `swap_ranges`, `iter_swap`	exchange
`equal(b, e, b2)`	ranges equal
`lexicographical_compare`	dictionary order
`max({a, b, c})`, `min({...})`	max/min of a list
`gcd`, `lcm` (numeric)	C++17
`inner_product`	dot product
`shuffle(b, e, rng)`	random order

3. STL Containers

3.0 Container cheat sheet

Container	Access	Insert/erase	Notes
`vector`	O(1)	O(1) back, O(n) middle	default choice
`deque`	O(1)	O(1) both ends	push_front allowed
`set`, `map`	O(log n)	O(log n)	sorted, lower_bound
`multiset`	O(log n)	O(log n)	duplicates allowed
`unordered_*`	O(1) avg	O(1) avg	hackable, no order
`priority_queue`	top O(1)	O(log n)	max heap default
`stack`, `queue`	top/front	O(1)	no iteration

3.1 vector

```
vector<int> a(n, 0);                    // n zeros
vector<vector<int>> g2(n, vector<int>(m, 0));
a.push_back(x); a.pop_back();
a.back(); a.front(); a.size(); a.empty();
a.insert(a.begin() + i, x);             // O(n)
a.erase(a.begin() + i);                 // erase one, O(n)
a.erase(a.begin() + l, a.begin() + r);// erase [l, r)
a.resize(k); a.assign(k, 0); a.clear();
vector<int> sub(a.begin() + l, a.begin() + r);   // slice
a.insert(a.end(), w.begin(), w.end());           // append
vector<int> c = a;                      // deep copy
swap(a, w);                             // O(1)
```

3.2 pair, tuple, array

```
pair<int, string> p1 = {1, "a"};
p1.first; p1.second; auto p2 = make_pair(1, 2);
auto [i1, s1] = p1;
tuple<int, int, int> tp = {1, 2, 3};
get<0>(tp); auto [t1, t2, t3] = tp;
array<int, 3> ar = {1, 2, 3};           // fixed size
vector<array<int, 3>> es;               // handy edge list
es.push_back({1, 2, 3});
tie(a, b) = make_pair(b, a);            // swap via tie
// pairs compare lexicographically: sort/min/max work
```

3.3 stack, queue, deque

```
stack<int> stk; stk.push(x); stk.top(); stk.pop();
queue<int> q; q.push(x); q.front(); q.back(); q.pop();
deque<int> dq;
dq.push_back(x); dq.push_front(x);
dq.pop_back(); dq.pop_front();
dq.front(); dq.back(); dq[i];         // random access ok
// all: size(), empty(). No clear(); assign a new object
```

3.4 priority_queue

```
priority_queue<int> mxh;                          // max heap
priority_queue<int, vector<int>, greater<int>> mnh;  // min
mxh.push(x); mxh.top(); mxh.pop(); mxh.size();
priority_queue<pii, vector<pii>, greater<pii>> pq;
// custom order: min heap by second
auto cmp = [](const pii& a, const pii& b) {
    return a.second > b.second;
};
priority_queue<pii, vector<pii>, decltype(cmp)> pq2(cmp);
// trick: push -x into a max heap to get a min heap
```

3.5 set and multiset

```
set<int> st2;
st2.insert(x); st2.erase(x); st2.count(x);
auto it = st2.find(x);                // end() if absent
if (it != st2.end()) st2.erase(it);
*st2.begin(); *st2.rbegin();          // min and max
auto lo = st2.lower_bound(x);         // first >= x
auto hi = st2.upper_bound(x);         // first > x
if (lo != st2.begin()) { auto pv = prev(lo); } // last < x
for (int e : st2) { }                 // sorted order
set<int, greater<int>> desc;          // descending
multiset<int> ms;
ms.erase(ms.find(x));                 // removes ONE copy
ms.erase(x);                          // removes ALL copies
ms.count(x);
```

3.6 map and unordered_map

```
map<string, int> mp2;
// creates entry 0 if absent
mp2["a"]++;
mp2.count("a"); mp2.erase("a");
auto it = mp2.find("a");          // it->first, it->second
for (auto& [key, val] : mp2) { }  // sorted by key
mp2.lower_bound("b");             // like set
mp2.begin()->first; mp2.rbegin()->first;
// at() throws if absent, [] inserts silently
unordered_map<int, int> um;
um.reserve(1 << 20);
```

Anti-hash safe `unordered_map` (plain ones can be hacked on Codeforces; fine on ICPC):

```
struct SplitHash {
    static ull mix(ull x) {
        x += 0x9e3779b97f4a7c15;
        x = (x ^ (x >> 30)) * 0xbf58476d1ce4e5b9;
        x = (x ^ (x >> 27)) * 0x94d049bb133111eb;
        return x ^ (x >> 31);
    }
    size_t operator()(ull x) const {
        static const ull R = chrono::steady_clock::now()
            .time_since_epoch().count();
        return mix(x + R);
    }
};
unordered_map<ll, int, SplitHash> safeMap;
```

3.7 bitset and iterators

```
// size fixed at compile time
bitset<64> bs;
bs[3] = 1; bs.set(5); bs.reset(5); bs.flip(2);
bs.count(); bs.any(); bs.none(); bs.test(3);
bs << 2; bs >> 1; bs & bs; bs | bs; bs ^ bs;
bs.to_ulong(); bs.to_string();
auto it = v.begin();
next(it, 2); prev(it, 1); advance(it, 3);
distance(v.begin(), it);          // index of an iterator
```

4. STL Algorithms

4.1 Sorting and comparators

```
sort(all(v));                             // ascending
sort(all(v), greater<int>());             // descending
sort(v.rbegin(), v.rend());               // descending too
stable_sort(all(v));                      // keeps equal order
vector<pii> pv(5);
// second DESC, first ASC (tie break)
sort(all(pv), [](const pii& a, const pii& b) {
    if (a.second != b.second)
        return a.second > b.second;
    return a.first < b.first;
});
reverse(all(v));
// sort indices by value
vector<int> idx(n); iota(all(idx), 0);
sort(all(idx), [&](int i, int j) { return v[i] < v[j]; });
is_sorted(all(v));
// v[k] correct
nth_element(v.begin(), v.begin() + k, v.end());
partial_sort(v.begin(), v.begin() + k, v.end());
```

Comparator must be a strict weak order: use < or >, never <= or >= (can crash).

4.2 Binary search: lower_bound, upper_bound

```
// v must be sorted ascending
// first >= t
int lb = lower_bound(all(v), target) - v.begin();
// first > t
int ub = upper_bound(all(v), target) - v.begin();
// occurrences of target
int cntEq = ub - lb;
bool has = binary_search(all(v), target);
auto [lo, hi] = equal_range(all(v), target);
// last element <= target
int lastLE = ub - 1;
// last element < target
int firstLT = lb - 1;
// descending vector: lower_bound(all(v), x, greater<int>())
// result == v.size() (or -1 above) means "not found"
```

4.3 Sum, accumulate, prefix sums, fill

```
// 0LL avoids overflow
ll sum = accumulate(all(v), 0LL);
ll prod = accumulate(all(v), 1LL, multiplies<ll>());
int mx2 = accumulate(all(v), INT_MIN,
    [](int a, int b) { return max(a, b); });
// vl[i] = v[0] + .. + v[i]
partial_sum(all(v), vl.begin());
// w[i] = v[i] - v[i-1]
adjacent_difference(all(v), w.begin());
iota(all(v), 1);                          // 1, 2, 3, ...
fill(all(v), 0);
inner_product(all(v), w.begin(), 0LL);  // dot product
ll s2 = reduce(all(v), 0LL);            // C++17, any order
// subarray sum with prefix array: see 8.8
```

4.4 Find, count, min, max

```
int cnt = count(all(v), x);                  // occurrences
int ev = count_if(all(v), [](int e) { return e % 2 == 0; });
// v.size() if absent
int f = find(all(v), x) - v.begin();
auto it = find_if(all(v), [](int e) { return e > 5; });
int maxVal = *max_element(all(v));
int minVal = *min_element(all(v));
// first max
int maxInd = max_element(all(v)) - v.begin();
int minInd = min_element(all(v)) - v.begin();
auto [mnIt, mxIt] = minmax_element(all(v));
bool a1 = any_of(all(v), [](int e) { return e < 0; });
bool a2 = all_of(all(v), [](int e) { return e > 0; });
bool a3 = none_of(all(v), [](int e) { return e == 0; });
// membership: sorted vector -> binary_search, set -> count
```

4.5 Remove, erase, unique, replace

```
// remove ALL occurrences of x (erase-remove idiom)
v.erase(remove(all(v), x), v.end());
// remove by condition
v.erase(remove_if(all(v), [](int e) { return e < 0; }),
        v.end());
v.erase(v.begin() + 2);                  // remove by index
auto it = find(all(v), x);               // remove FIRST x
if (it != v.end()) v.erase(it);
sort(all(v));                            // distinct values
v.erase(unique(all(v)), v.end());
// every 3 becomes 9
replace(all(v), 3, 9);
replace_if(all(v), [](int e) { return e < 0; }, 0);
// rotate left by 2
rotate(v.begin(), v.begin() + 2, v.end());
// string: s.erase(remove(all(s), ' '), s.end());
// C++20 only: erase(v, x); erase_if(v, pred);
```

4.6 Permutations, subsets, combinations

```
sort(all(v));
do {
    // v holds one arrangement (duplicates handled)
} while (next_permutation(all(v)));
prev_permutation(all(v));
// all subsets with a bitmask (n up to 20)
for (int mask = 0; mask < (1 << n); mask++)
    for (int i = 0; i < n; i++)
        if (mask >> i & 1) { /* v[i] is in the subset */ }
// k-combinations: permute a 0/1 selector
vector<int> sel(n, 0);
fill(sel.end() - k, sel.end(), 1);
do { /* sel[i] == 1 means v[i] chosen */ }
while (next_permutation(all(sel)));
```

```
void permute(vector<int>& arr, vector<int>& cur,
             vector<bool>& used) {
    if (cur.size() == arr.size()) {
        for (int x : cur) cout << x << " ";
        cout << "\n";
        return;
    }
    for (int i = 0; i < (int)arr.size(); i++) {
        if (used[i]) continue;
        used[i] = true;
        cur.push_back(arr[i]);
        permute(arr, cur, used);
        cur.pop_back();
        used[i] = false;
    }
}
// call: vector<int> cur; vector<bool> used(n);
//       permute(arr, cur, used);
```

4.7 Set operations and merging (sorted input)

```
vector<int> out;
set_intersection(all(v), all(w), back_inserter(out));
set_union(all(v), all(w), back_inserter(out));
set_difference(all(v), all(w), back_inserter(out));
merge(all(v), all(w), back_inserter(out));
bool sub = includes(all(v), all(w));    // is w inside v
```

4.8 GCD, LCM, min, max, math helpers

```
int g1 = gcd(a, b);  int l1 = lcm(a, b);       // C++17
ll g2 = __gcd(p, q);                            // older
// lcm w/o overflow
ll l2 = p / __gcd(p, q) * q;
int mn3 = min({a, b, c}); int mx3 = max({a, b, c});
swap(a, b);
int cl = clamp(x, 0, 10);
int ab = abs(x); ll ab2 = llabs(p);
double rt = sqrt(2.0); long double rt2 = sqrtl(2);
double pi = acos(-1.0);
double h = hypot(3.0, 4.0);
ll fl = floor(2.5); ll ce = ceil(2.5);
```

```
ll isqrt(ll n) {               // exact integer square root
    ll r = sqrtl(n);
    while (r * r > n) r--;
    while ((r + 1) * (r + 1) <= n) r++;
    return r;
}
```

4.9 Bit builtins

```
int oneCnt = __builtin_popcount(x);      // number of ones
// trailing zeros, x != 0
int tz = __builtin_ctz(x);
// leading zeros, x != 0
int lz = __builtin_clz(x);
// long long versions
int oneCntL = __builtin_popcountll(p);
// highest set bit index
int hb = 31 - __builtin_clz(x);
int lg = __lg(x);                        // floor(log2(x))
int par = __builtin_parity(x);
```

5. Strings

5.1 String operations

```
string s2 = "hello world";
s2.substr(pos, len); s2.substr(pos);     // len optional
s2.find("lo");                           // npos if absent
s2.find('o', 5); s2.rfind('o');
if (s2.find("zz") == string::npos) { }
s2.erase(pos, len); s2.insert(pos, "xx");
s2.replace(pos, len, "yy");
s2.push_back('c'); s2.pop_back(); s2 += "abc";
s2.front(); s2.back(); s2.size();
reverse(all(s2)); sort(all(s2));
s2 == t; s2 < t; s2.compare(t);          // lexicographic
// split on spaces
stringstream ss(s2); string w2; vector<string> parts;
while (ss >> w2) parts.push_back(w2);
// split on a delimiter
stringstream sd("a,b,c");
while (getline(sd, w2, ',')) parts.push_back(w2);
// letter frequency
int freq[26] = {};
for (char c : s2) if (islower(c)) freq[c - 'a']++;
// palindrome check
bool pal = equal(s2.begin(), s2.begin() + s2.size() / 2,
                 s2.rbegin());
// is a a subsequence of b (two pointers)
size_t j = 0;
for (char c : s2) if (j < t.size() && c == t[j]) j++;
bool isSub = (j == t.size());
```

5.2 Prefix function (KMP) and Z-function

```
vector<int> prefixFunction(const string& s) {
    int n = s.size();
    vector<int> pi(n, 0);
    for (int i = 1; i < n; i++) {
        int j = pi[i - 1];
        while (j > 0 && s[i] != s[j]) j = pi[j - 1];
        if (s[i] == s[j]) j++;
        pi[i] = j;
    }
    return pi;
}
// find pattern p in text t: pi of p + "#" + t,
// match ends where pi value == p.size()
vector<int> zFunction(const string& s) {
    int n = s.size();
    vector<int> z(n, 0);
    for (int i = 1, l = 0, r = 0; i < n; i++) {
        if (i < r) z[i] = min(r - i, z[i - l]);
        while (i + z[i] < n && s[z[i]] == s[i + z[i]])
            z[i]++;
        if (i + z[i] > r) l = i, r = i + z[i];
    }
    return z;
}
```

5.3 Rolling hash (mod 2^61 - 1)

```
struct Hash {
    static const ull M = (1ULL << 61) - 1;
    vector<ull> h, pw;
    static ull mul(ull a, ull b) {
        __uint128_t t = (__uint128_t)a * b;
        ull r = (t & M) + (t >> 61);
        return r >= M ? r - M : r;
    }
    Hash(const string& s, ull B = 911382323) {
        int n = s.size();
        h.assign(n + 1, 0); pw.assign(n + 1, 1);
        for (int i = 0; i < n; i++) {
            h[i + 1] = mul(h[i], B) + s[i];
            if (h[i + 1] >= M) h[i + 1] -= M;
            pw[i + 1] = mul(pw[i], B);
        }
    }
    ull get(int l, int r) {              // hash of s[l, r)
        ull x = h[r] + M - mul(h[l], pw[r - l]);
        return x >= M ? x - M : x;
    }
};
```

5.4 Trie

```
struct Trie {
    vector<array<int, 26>> nx;
    vector<int> cntEnd;
    Trie() { newNode(); }
    int newNode() {
        array<int, 26> a; a.fill(-1);
        nx.push_back(a); cntEnd.push_back(0);
        return nx.size() - 1;
    }
    void insert(const string& s) {
        int u = 0;
        for (char c : s) {
            int k = c - 'a';
            if (nx[u][k] < 0) {
                int v = newNode();
                nx[u][k] = v;
            }
            u = nx[u][k];
        }
        cntEnd[u]++;
    }
    bool find(const string& s) {
        int u = 0;
        for (char c : s) {
            u = nx[u][c - 'a'];
            if (u < 0) return false;
        }
        return cntEnd[u] > 0;
    }
};
```

6. Python Quick Reference

6.1 C++ and Python side by side

Task	C++	Python
sum	`accumulate(all(v),0LL)`	`sum(a)`
max / min	`*max_element(all(v))`	`max(a)`, `min(a)`
index of max	`max_element(all(v))-v.begin()`	`a.index(max(a))`
find index	`find(all(v),x)-v.begin()`	`a.index(x)` (error if absent)
exists	`count(all(v),x)>0`	`x in a`
count	`count(all(v),x)`	`a.count(x)`
remove all x	`v.erase(remove(all(v),x),v.end())`	`a=[y for y in a if y!=x]`
remove first x	`v.erase(find(all(v),x))`	`a.remove(x)`
remove at i	`v.erase(v.begin()+i)`	`a.pop(i)` or `del a[i]`
distinct sorted	`sort; erase(unique(..))`	`sorted(set(a))`
sort desc	`sort(all(v),greater<>())`	`a.sort(reverse=True)`
sort by key	`sort(all(v),lambda)`	`a.sort(key=lambda p: (-p[1],p[0]))`
reverse	`reverse(all(v))`	`a.reverse()` or `a[::-1]`
lower_bound	`lower_bound(..)-v.begin()`	`bisect_left(a,x)`
upper_bound	`upper_bound(..)-v.begin()`	`bisect_right(a,x)`
prefix sums	`partial_sum`	`list(accumulate(a))`
fill	`vector<int> a(n,0)`	`a=[0]*n`
2D array	`vector<vector<int>> g(n,vector<int>(m))`	`g=[[0]*m for _ in range(n)]`
permutations	`next_permutation`	`itertools.permutations`
gcd, lcm	`gcd(a,b)`, `lcm(a,b)`	`math.gcd`, `math.lcm`
mod power	`power(a,b,m)`	`pow(a,b,m)`
mod inverse	`power(a,m-2,m)`	`pow(a,-1,m)`
popcount	`__builtin_popcount(x)`	`x.bit_count()`
swap	`swap(a,b)`	`a,b=b,a`
min of 3	`min({a,b,c})`	`min(a,b,c)`
str to int	`stoi(s)`	`int(s)`
heap	`priority_queue`	`heapq` (min heap)
ordered set	`set`, `lower_bound`	no built-in (use sorted list + bisect)

6.2 Input, output, lists

```
import sys
input = sys.stdin.readline
n = int(input())
a = list(map(int, input().split()))
n, m = map(int, input().split())
s = input().strip()
grid = [input().strip() for _ in range(n)]
# read everything at once
data = sys.stdin.read().split()
print(*a)                              # space separated
print("\n".join(map(str, a)))
print(f"{x:.6f}")                      # 6 decimals
sys.stdout.write("hi\n")

a = [0] * n                            # 1D list
g = [[0] * m for _ in range(n)]        # 2D (never [[0]*m]*n)
a.append(x); a.pop(); a.insert(i, x)
a[l:r]; a[::2]; a[-1]; a[::-1]         # slices
a.extend(b); a = a + b
sum(a); min(a); max(a); len(a)
b = sorted(a); a.sort(reverse=True)
# second desc, first asc
a.sort(key=lambda p: (-p[1], p[0]))
```

6.3 Accumulate, find, count, remove

```
from itertools import accumulate
# pref[i] = sum of a[:i]
pref = [0] + list(accumulate(a))
rangeSum = pref[r] - pref[l]           # sum of a[l:r]
runMax = list(accumulate(a, max))      # running maximum
# first index, error if absent
a.index(x)
a.index(max(a))                        # index of the max
x in a                                 # O(n) membership
a.count(x)
# removes FIRST x, error if absent
a.remove(x)
a = [y for y in a if y != x]           # remove ALL x
del a[i]; del a[l:r]; a.pop(i)         # by index, by slice
a = sorted(set(a))                     # distinct and sorted
a = list(dict.fromkeys(a))             # distinct, keeps order
pos = [i for i, y in enumerate(a) if y == x]   # all indices
for i, y in enumerate(a): pass
for x1, y1 in zip(a, b): pass
any(y < 0 for y in a); all(y > 0 for y in a)
sum(1 for y in a if y % 2 == 0)        # count_if
```

6.4 dict, set, Counter, deque

```
from collections import Counter, defaultdict, deque
d = {}
d[k] = d.get(k, 0) + 1
d = defaultdict(int); d = defaultdict(list)
c = Counter(a); c.most_common(3); c[x]
for k, v in d.items(): pass
d.pop(k, None); del d[k]; k in d
sorted(d.items(), key=lambda kv: kv[1])
s = set(a)
# remove: error if absent
s.add(x); s.discard(x); s.remove(x)
s | t; s & t; s - t; s ^ t
dq = deque()
dq.append(x); dq.appendleft(x); dq.pop(); dq.popleft()
```

6.5 heapq and bisect

```
import heapq, bisect
h = []
heapq.heappush(h, x); heapq.heappop(h); h[0]   # min heap
# max heap: store negatives
heapq.heappush(h, -x)
heapq.heapify(a)
heapq.nsmallest(k, a); heapq.nlargest(k, a)
bisect.bisect_left(a, x)             # C++ lower_bound
bisect.bisect_right(a, x)            # C++ upper_bound
bisect.insort(a, x)                  # keep list sorted
```

6.6 Strings

```
s.split(); s.split(","); " ".join(words); s.strip()
s.find("x")                          # -1 if absent
s.index("x")                         # error if absent
s.count("ab"); s.replace("a", "b"); s.startswith("ab")
s.upper(); s.lower(); s.isdigit(); s.isalpha()
ord("a"); chr(97); int("101", 2); bin(x); hex(x)
format(x, "b"); format(x, "08b")     # binary, zero padded
s[::-1]; "".join(sorted(s)); s == s[::-1]
c = ord(ch) - ord("a")               # letter index
```

6.7 math, itertools, functools, bits

```
import math
from functools import lru_cache, reduce, cmp_to_key
from itertools import permutations, combinations, product
math.gcd(a, b); math.lcm(a, b); math.isqrt(n)
math.comb(n, r); math.perm(n, r); math.factorial(n)
# modular inverse (3.8+)
pow(a, b, MOD); pow(a, -1, MOD)
math.inf; math.log2(x); math.ceil(x); math.floor(x)
# a // b floors toward -inf
divmod(a, b)
list(permutations(a)); list(combinations(a, 2))
list(product(a, repeat=2))
@lru_cache(maxsize=None)
def f(i, j): pass
reduce(lambda x, y: x * y, a, 1)
a.sort(key=cmp_to_key(lambda x, y: x - y))
x & 1; x >> k & 1; x | (1 << k); x & ~(1 << k); x ^ (1 << k)
x & -x; x.bit_length(); x.bit_count()   # bit_count: 3.10+
```

6.8 Graph code in Python

```
import heapq
from collections import deque
adj = [[] for _ in range(n + 1)]
for _ in range(m):
    u, v = map(int, input().split())
    adj[u].append(v); adj[v].append(u)

def bfs(src):
    dist = [-1] * (n + 1)
    dist[src] = 0
    q = deque([src])
    while q:
        u = q.popleft()
        for v in adj[u]:
            if dist[v] == -1:
                dist[v] = dist[u] + 1
                q.append(v)
    return dist

# wadj[u] = [(v, w), ...]
def dijkstra(src, wadj):
    dist = [float("inf")] * (n + 1)
    dist[src] = 0
    pq = [(0, src)]
    while pq:
        d, u = heapq.heappop(pq)
        if d > dist[u]:
            continue
        for v, w in wadj[u]:
            if d + w < dist[v]:
                dist[v] = d + w
                heapq.heappush(pq, (d + w, v))
    return dist

# no recursion limit issues
def dfs_iter(src):
    seen = [False] * (n + 1)
    st = [src]
    while st:
        u = st.pop()
        if seen[u]:
            continue
        seen[u] = True
        for v in adj[u]:
            if not seen[v]:
                st.append(v)
    return seen
```

6.9 Python speed tips

```
- Read with sys.stdin.readline or sys.stdin.read().split()
- Print once: "\n".join(map(str, res)), not print in a loop
- Prefer list comprehensions and builtins (sum, min, sorted)
- Recursion: sys.setrecursionlimit, but iterative is safer
- Submit as PyPy when allowed; avoid numpy tricks there
- Big integers are automatic: no overflow, but slower
- a.pop(0) and a.insert(0, x) are O(n): use deque
```

6.10 Python input patterns (every format)

```
import sys
input = sys.stdin.readline        # fast; keeps "\n" at end

n = int(input())                  # one integer
n, m = map(int, input().split())  # two integers
a = list(map(int, input().split()))   # list on one line
x = float(input())                # a float
s = input().strip()               # string (strip the \n!)
line = input().rstrip("\n")       # line, keep inner spaces
chars = list(input().strip())     # string as list of chars
digits = list(map(int, input().strip()))  # "12345"
words = input().split()           # list of words

# T test cases
T = int(input())
for _ in range(T):
    n = int(input())
    a = list(map(int, input().split()))

# n numbers, one per line
a = [int(input()) for _ in range(n)]

# n pairs / edges "u v" or "u v w"
pairs = [tuple(map(int, input().split())) for _ in range(m)]
for u, v in pairs:
    pass

# matrix of ints (n rows)
mat = [list(map(int, input().split())) for _ in range(n)]

# grid of chars, access grid[i][j]
grid = [input().strip() for _ in range(n)]
# editable
grid2 = [list(input().strip()) for _ in range(n)]

# queries with a type: "1 x y" or "2 x"
q = int(input())
for _ in range(q):
    t, *rest = map(int, input().split())
    if t == 1:
        x, y = rest
    else:
        (x,) = rest

# first number then the list on the same line: "3 7 8 9"
k, *vals = map(int, input().split())

# comma separated "3,4,5"
nums = list(map(int, input().split(",")))
```

6.11 Python input: tokens, EOF, huge input

Each block below is a separate reading style. Pick one per program: they all consume stdin.

```
import sys

# read ALL tokens at once (fastest, ignores line breaks)
data = sys.stdin.buffer.read().split()
idx = 0
def nxt():
    global idx
    idx += 1
    return data[idx - 1]
n = int(nxt())
a = [int(nxt()) for _ in range(n)]
s = nxt().decode()                # bytes to str

# same idea with an iterator
it = iter(sys.stdin.read().split())
n = int(next(it))
a = [int(next(it)) for _ in range(n)]

# read until EOF, line by line
for line in sys.stdin:
    line = line.strip()
    if not line:
        continue
    nums = list(map(int, line.split()))

# all lines at once
lines = sys.stdin.read().splitlines()

# input() raises EOFError at end of file
while True:
    try:
        line = input()
    except EOFError:
        break

# output fast
out = []
out.append(str(42))
sys.stdout.write("\n".join(out) + "\n")
print(*a, sep=" ")                # 1 2 3
print(*a, sep="\n")               # one per line
print("Case #%d: %d" % (1, 42))
print(f"Case #{1}: {42}")
print(f"{3.14159:.3f}")           # 3.142
print("YES" if True else "NO")
print(" ".join(map(str, a)))
for row in grid2:
    print("".join(row))
print(end="")                     # no newline
```

6.12 Python syntax essentials

```
# variables, multiple assignment, swap
a, b = 1, 2
a, b = b, a
x = y = 0
first, *middle, last = [1, 2, 3, 4]

# arithmetic
7 // 2      # 3   floor division
-7 // 2     # -4  floors toward -inf (C++ gives -3)
7 % 3       # 1
-7 % 3      # 2   always same sign as divisor
2 ** 10     # 1024
divmod(17, 5)       # (3, 2)
abs(-3); round(2.5) # round(2.5) == 2 (banker's rounding)
int(3.9)    # 3 truncates
10 ** 18 + 1        # big ints never overflow
float("inf"); -float("inf")

# comparison chaining and logic
0 <= x < n
x == 1 and y == 2 or not (x > 3)
x is None

# if / elif / else, ternary
if x > 0:
    pass
elif x == 0:
    pass
else:
    pass
sign = 1 if x >= 0 else -1

# loops
for i in range(n): pass            # 0..n-1
for i in range(1, n + 1): pass     # 1..n
for i in range(n - 1, -1, -1): pass   # reverse
for i in range(0, n, 2): pass      # step 2
for i, v in enumerate(a): pass
for i, v in enumerate(a, start=1): pass
for u, v in zip(a, b): pass
for k, v in d.items(): pass
while x > 0:
    x //= 10
# for ... else: else runs if the loop did NOT break
for v in a:
    if v < 0:
        break
else:
    print("no negatives")
```

6.13 Functions, lambdas, recursion

```
import sys
from functools import lru_cache, cache

def add(a, b=0):                  # default argument
    return a + b

def stats(arr):                   # return several values
    return min(arr), max(arr), sum(arr)
lo, hi, total = stats([3, 1, 2])

def total_all(*args, **kwargs):   # any number of args
    return sum(args)

# small anonymous function
sq = lambda v: v * v
a.sort(key=lambda p: (p[1], -p[0]))

# modify outer variables inside a nested function
def outer():
    cnt = 0
    def inc():
        nonlocal cnt
        cnt += 1
    inc()
    return cnt

best = 0
def update(v):
    global best
    best = max(best, v)

# lists are passed by reference: changes are visible
def push(lst):
    lst.append(1)

# memoized recursion
sys.setrecursionlimit(10 ** 6)
@lru_cache(maxsize=None)          # or @cache (3.9+)
def fib(k):
    return k if k < 2 else fib(k - 1) + fib(k - 2)
fib.cache_clear()                 # between test cases

# deep recursion: run inside a thread with a big stack
import threading
def main():
    pass
threading.stack_size(1 << 27)
th = threading.Thread(target=main)
th.start(); th.join()
```

6.14 Classes, dataclasses, sorting objects

```
from dataclasses import dataclass
from functools import cmp_to_key

class DSU:
    def __init__(self, n):
        self.p = list(range(n + 1))
        self.sz = [1] * (n + 1)
    def find(self, x):
        while self.p[x] != x:
            self.p[x] = self.p[self.p[x]]   # path halving
            x = self.p[x]
        return x
    def union(self, a, b):
        a, b = self.find(a), self.find(b)
        if a == b:
            return False
        if self.sz[a] < self.sz[b]:
            a, b = b, a
        self.p[b] = a
        self.sz[a] += self.sz[b]
        return True
```

```
# order=True gives < for sorting
@dataclass(order=True)
class Job:
    deadline: int
    profit: int

jobs = [Job(2, 10), Job(1, 5)]
jobs.sort()                  # by deadline, then profit

class Point:
    def __init__(self, x, y):
        self.x, self.y = x, y
    # used by sort and heapq
    def __lt__(self, other):
        return (self.x, self.y) < (other.x, other.y)
    def __repr__(self):
        return f"({self.x}, {self.y})"

# custom comparator like C++ (negative means a first)
def cmp(a, b):
    return (a + b > b + a) - (a + b < b + a)
words = ["9", "34", "3"]
words.sort(key=cmp_to_key(cmp), reverse=True)  # "9343"

# tuples are the easiest "struct": sort by fields
people = [("bob", 25), ("amy", 25), ("cat", 20)]
# age desc, name asc
people.sort(key=lambda t: (-t[1], t[0]))
```

6.15 Comprehensions, slicing, unpacking

```
sq = [x * x for x in range(10)]
evens = [x for x in a if x % 2 == 0]
pairs = [(i, j) for i in range(3) for j in range(i)]
flat = [v for row in mat for v in row]
d = {x: i for i, x in enumerate(a)}       # value -> index
s = {x % 3 for x in a}
g = sum(x for x in a if x > 0)            # generator
cols = list(zip(*mat))                    # transpose
rot = [list(r) for r in zip(*mat[::-1])]  # rotate 90 cw

a[i]; a[-1]; a[-2]               # last, second last
a[l:r]                           # l .. r-1
a[:k]; a[k:]                     # first k, rest
a[::-1]                          # reversed copy
a[::2]                           # every other
b = a[:]                         # shallow copy
import copy
c = copy.deepcopy(mat)           # full copy of 2D
a[l:r] = [0] * (r - l)           # assign a slice

# NEVER: g = [[0] * m] * n  (all rows are the same list)
g = [[0] * m for _ in range(n)]
```

6.16 Python method catalog

Call	Does
`a.append(x)`, `a.extend(b)`	add one, add many
`a.insert(i, x)`	insert at i, O(n)
`a.pop()`, `a.pop(i)`	remove last / at i, returns it
`a.remove(x)`	delete first x (ValueError if none)
`a.index(x)`	first index (ValueError if none)
`a.count(x)`	occurrences
`a.sort()`, `sorted(a)`	in place / new list
`a.reverse()`, `reversed(a)`	in place / iterator
`a.clear()`, `a.copy()`	empty, shallow copy
`len`, `sum`, `min`, `max`	builtins, take key= too
`max(a, key=len)`, `min(d, key=d.get)`	arg max, key with min value
`any(...)`, `all(...)`	bool tests
`map(f, a)`, `filter(f, a)`	lazy apply / keep
`zip(a, b)`, `enumerate(a)`	pair up, index + value
`range(l, r, step)`	integer sequence
`abs`, `pow(a, b, m)`, `divmod`	math builtins
`ord(c)`, `chr(n)`	char to code and back
`int(s, base)`, `bin`, `hex`, `oct`	base conversion
`str(x)`, `repr(x)`	to string
`d.get(k, default)`	value or default
`d.setdefault(k, [])`	get or insert default
`d.keys()`, `d.values()`, `d.items()`	views
`d.pop(k)`, `del d[k]`	remove key
`s.add(x)`, `s.discard(x)`	set add / remove safely
`s.union(t)`, `s.intersection(t)`	set algebra
`s.issubset(t)`, `s.isdisjoint(t)`	set tests
`st.split()`, `" ".join(list)`	split / join strings
`st.strip()`, `st.lstrip()`, `st.rstrip()`	trim
`st.find(t)`, `st.rfind(t)`	index or -1
`st.count(t)`, `st.replace(a, b)`	count, replace all
`st.startswith(t)`, `st.endswith(t)`	prefix / suffix
`st.isdigit()`, `st.isalpha()`, `st.isalnum()`	char class
`st.upper()`, `st.lower()`, `st.swapcase()`	case
`st.zfill(5)`, `st.center(9, "*")`	padding
`st[::-1]`, `st * 3`	reverse, repeat
`Counter(a).most_common(k)`	top k frequent
`deque.rotate(k)`	rotate right by k
`heapq.heappushpop(h, x)`	push then pop min
`bisect.insort(a, x)`	insert keeping sorted
`math.isqrt`, `math.comb`, `math.gcd`	exact integer math
`itertools.accumulate`, `.product`	prefix, cartesian
`itertools.groupby(sorted(a))`	runs of equal values
`itertools.combinations(a, k)`	k subsets
`itertools.permutations(a, k)`	k arrangements
`itertools.chain(a, b)`	concatenate iterables

7. Graphs

7.1 Building a graph

```
int N, M; cin >> N >> M;
// undirected, unweighted, 1-indexed
vector<vector<int>> g1(N + 1);
for (int i = 0; i < M; i++) {
    int u, v; cin >> u >> v;
    g1[u].push_back(v);
    // delete this line if directed
    g1[v].push_back(u);
}
// weighted: store {neighbor, weight}
vector<vector<pair<int, ll>>> wg(N + 1);
for (int i = 0; i < M; i++) {
    int u, v; ll w; cin >> u >> v >> w;
    wg[u].push_back({v, w});
    wg[v].push_back({u, w});
}
// edge list (Kruskal, Bellman-Ford)
vector<array<ll, 3>> es;      // {w, u, v}
for (int i = 0; i < M; i++) {
    int u, v; ll w; cin >> u >> v >> w;
    es.push_back({w, u, v});
}
// adjacency matrix (Floyd Warshall, n <= 500)
vector<vector<ll>> mat(N + 1, vector<ll>(N + 1, INF));
// 0-indexed input: read u--, v-- and use size N
// tree: M = N - 1 edges, same code
// grid as a graph: see 7.2 (4 directions dx, dy)
```

7.2 BFS: distances, grid, multi-source

```
vector<int> bfs(int start, vector<vector<int>>& adj) {
    int n = adj.size();
    vector<int> dist(n, -1);
    queue<int> q;
    q.push(start);
    dist[start] = 0;
    while (!q.empty()) {
        int u = q.front();
        q.pop();
        for (int v : adj[u]) {
            if (dist[v] != -1) continue;
            dist[v] = dist[u] + 1;
            q.push(v);
        }
    }
    return dist;          // -1 means unreachable
}
// multi-source BFS: push all sources first with dist 0
```

```
const int dx[] = {1, -1, 0, 0}, dy[] = {0, 0, 1, -1};
// 8 directions: dx = {1,1,1,0,0,-1,-1,-1}
//               dy = {1,0,-1,1,-1,1,0,-1}
int gridBfs(vector<string>& g, int sx, int sy,
            int tx, int ty) {
    int R = g.size(), C = g[0].size();
    vector<vector<int>> d(R, vector<int>(C, -1));
    queue<pair<int, int>> q;
    q.push({sx, sy}); d[sx][sy] = 0;
    while (!q.empty()) {
        auto [x, y] = q.front(); q.pop();
        for (int k = 0; k < 4; k++) {
            int nx = x + dx[k], ny = y + dy[k];
            if (nx < 0 || ny < 0 || nx >= R || ny >= C)
                continue;
            if (g[nx][ny] == '#' || d[nx][ny] != -1)
                continue;
            d[nx][ny] = d[x][y] + 1;
            q.push({nx, ny});
        }
    }
    return d[tx][ty];     // -1 if unreachable
}
// path: keep parent[v] = u, then walk back from t
```

7.3 DFS, components, cycles, topological sort

```
void dfs(int u, vector<vector<int>>& adj,
         vector<int>& vis) {
    vis[u] = 1;
    for (int v : adj[u]) {
        if (!vis[v])
            dfs(v, adj, vis);
    }
}
// count components: for each unvisited u: dfs, cnt++
// very deep recursion (1e5+ nodes): go iterative
```

```
// undirected cycle (no multi-edges): call cyc(u, 0, ...)
bool cyc(int u, int p, vector<vector<int>>& g,
         vector<int>& vis) {
    vis[u] = 1;
    for (int v : g[u]) {
        if (v == p) continue;
        if (vis[v] || cyc(v, u, g, vis)) return true;
    }
    return false;
}
// directed cycle: color 0 = new, 1 = on stack, 2 = done
vector<int> color;
bool hasCycle(int u, vector<vector<int>>& g) {
    color[u] = 1;
    for (int v : g[u]) {
        if (color[v] == 1) return true;
        if (color[v] == 0 && hasCycle(v, g)) return true;
    }
    color[u] = 2;
    return false;
}
// topological order (Kahn). order.size() < n means a cycle
vector<int> topoSort(int n, vector<vector<int>>& g) {
    vector<int> indeg(n + 1), order;
    for (int u = 1; u <= n; u++)
        for (int v : g[u]) indeg[v]++;
    queue<int> q;
    for (int u = 1; u <= n; u++)
        if (!indeg[u]) q.push(u);
    while (!q.empty()) {
        int u = q.front(); q.pop();
        order.push_back(u);
        for (int v : g[u])
            if (--indeg[v] == 0) q.push(v);
    }
    return order;
}
```

7.4 Shortest paths

```
vector<vector<pair<int, ll>>> adj(100005);   // {to, weight}
int n;
vector<ll> dijkstra(int src) {       // non-negative weights
    priority_queue<pair<ll, int>, vector<pair<ll, int>>,
                   greater<>> pq;
    vector<ll> dist(n + 1, INF);
    dist[src] = 0;
    pq.push({0, src});
    while (!pq.empty()) {
        auto [d, u] = pq.top();
        pq.pop();
        if (d > dist[u]) continue;
        for (auto [v, w] : adj[u]) {
            if (dist[v] > d + w) {
                dist[v] = d + w;
                pq.push({dist[v], v});
            }
        }
    }
    // INF means unreachable
    return dist;
}
// path: keep par[v] = u when relaxing, walk back
```

```
vector<vector<ll>> floydWarshall() {         // n <= ~500
    vector<vector<ll>> d(n + 1, vector<ll>(n + 1, INF));
    for (int i = 1; i <= n; i++) d[i][i] = 0;
    for (int u = 1; u <= n; u++)
        for (auto [v, w] : adj[u])
            d[u][v] = min(d[u][v], w);
    for (int k = 1; k <= n; k++) {
        for (int i = 1; i <= n; i++) {
            if (d[i][k] == INF) continue;
            for (int j = 1; j <= n; j++) {
                if (d[k][j] == INF) continue;
                d[i][j] = min(d[i][j], d[i][k] + d[k][j]);
            }
        }
    }
    // d[i][i] < 0: negative cycle
    return d;
}
```

```
struct Edge { int u, v; ll w; };
vector<Edge> edges;
// Bellman-Ford: true if a negative cycle is reachable
bool bellmanFord(int src, vector<ll>& dist) {
    dist.assign(n + 1, INF);
    dist[src] = 0;
    for (int i = 0; i < n; i++) {
        bool changed = false;
        for (auto& e : edges) {
            if (dist[e.u] != INF &&
                dist[e.u] + e.w < dist[e.v]) {
                dist[e.v] = dist[e.u] + e.w;
                changed = true;
            }
        }
        if (!changed) return false;
        if (i == n - 1) return true;
    }
    return false;
}
// 0-1 BFS: weights 0 or 1, deque instead of heap, O(n + m)
vector<int> bfs01(int src) {
    vector<int> dist(n + 1, INT_MAX);
    deque<int> dq;
    dist[src] = 0; dq.push_back(src);
    while (!dq.empty()) {
        int u = dq.front(); dq.pop_front();
        for (auto [v, w] : adj[u]) {
            if (dist[u] + w < dist[v]) {
                dist[v] = dist[u] + w;
                if (w == 0) dq.push_front(v);
                else dq.push_back(v);
            }
        }
    }
    return dist;
```

```
}
```

7.5 DSU and minimum spanning tree

```
struct DSU {
    vector<int> p, siz;
    DSU(int n) : p(n + 1), siz(n + 1, 1) {
        iota(p.begin(), p.end(), 0);
    }
    int find(int x) {
        return p[x] == x ? x : p[x] = find(p[x]);
    }
    bool unite(int a, int b) {
        a = find(a); b = find(b);
        if (a == b) return false;
        if (siz[a] < siz[b]) swap(a, b);
        p[b] = a; siz[a] += siz[b];
        return true;
    }
};
// Kruskal: es = {w, u, v}
ll kruskal(int n, vector<array<ll, 3>> es) {
    sort(es.begin(), es.end());
    DSU d(n);
    ll total = 0;
    for (auto [w, u, v] : es)
        if (d.unite(u, v)) total += w;
    return total;     // check all joined: n - 1 edges used
}
// Prim with a heap (adj is {to, weight})
ll prim(int n, vector<vector<pair<int, ll>>>& adj) {
    vector<bool> vis(n + 1, false);
    priority_queue<pair<ll, int>, vector<pair<ll, int>>,
                   greater<>> pq;
    ll total = 0;
    pq.push({0, 1});
    while (!pq.empty()) {
        auto [w, u] = pq.top(); pq.pop();
        if (vis[u]) continue;
        vis[u] = true; total += w;
        for (auto [v, c] : adj[u])
            if (!vis[v]) pq.push({c, v});
    }
    return total;
}
```

7.6 Bridges, articulation points, SCC

```
vector<vector<pair<int, int>>> g2;   // {to, edge id}
vector<int> tin, low;
vector<bool> isBridge;
int timer_ = 0;
void dfsBridge(int u, int parentEdge) {
    tin[u] = low[u] = ++timer_;
    for (auto [v, id] : g2[u]) {
        if (id == parentEdge) continue;
        if (tin[v]) low[u] = min(low[u], tin[v]);
        else {
            dfsBridge(v, id);
            low[u] = min(low[u], low[v]);
            if (low[v] > tin[u]) isBridge[id] = true;
        }
    }
}
// init: tin, low = vector<int>(n + 1, 0)
// articulation points on a plain adjacency list g
vector<vector<int>> g;
vector<bool> cut;
void dfsCut(int u, int p) {
    tin[u] = low[u] = ++timer_;
    int children = 0;
    for (int v : g[u]) {
        if (v == p) continue;
        if (tin[v]) low[u] = min(low[u], tin[v]);
        else {
            dfsCut(v, u);
            low[u] = min(low[u], low[v]);
            if (low[v] >= tin[u] && p != -1) cut[u] = true;
            children++;
        }
    }
    if (p == -1 && children > 1) cut[u] = true;
}
// call dfsCut(root, -1) for each unvisited root
```

```
// Strongly connected components (Kosaraju)
// graph and reversed graph
vector<vector<int>> gf, gr;
vector<int> ord, comp;
vector<bool> used;
void dfs1(int u) {
    used[u] = true;
    for (int v : gf[u]) if (!used[v]) dfs1(v);
    ord.push_back(u);
}
void dfs2(int u, int c) {
    comp[u] = c;
    for (int v : gr[u]) if (comp[v] == -1) dfs2(v, c);
}
int scc(int n) {
    used.assign(n + 1, false);
    comp.assign(n + 1, -1);
    ord.clear();
    for (int u = 1; u <= n; u++) if (!used[u]) dfs1(u);
    int c = 0;
    for (int i = n - 1; i >= 0; i--)
        if (comp[ord[i]] == -1) dfs2(ord[i], c++);
    // comp[u] = component id
    return c;
}
```

7.7 Trees: subtree size, diameter, LCA

```
const int LOG = 20;
vector<vector<int>> tr;
int up[200005][LOG], dep[200005], sub[200005];
void dfsLca(int u, int p) {
    up[u][0] = p;
    sub[u] = 1;
    for (int j = 1; j < LOG; j++)
        up[u][j] = up[up[u][j - 1]][j - 1];
    for (int v : tr[u]) {
        if (v == p) continue;
        dep[v] = dep[u] + 1;
        dfsLca(v, u);
        sub[u] += sub[v];            // subtree size
    }
}
// setup: dep[1] = 0; dfsLca(1, 1);  (root's parent is root)
int lca(int a, int b) {
    if (dep[a] < dep[b]) swap(a, b);
    int d = dep[a] - dep[b];
    for (int j = 0; j < LOG; j++)
        if (d >> j & 1) a = up[a][j];
    if (a == b) return a;
    for (int j = LOG - 1; j >= 0; j--)
        if (up[a][j] != up[b][j]) {
            a = up[a][j];
            b = up[b][j];
        }
    return up[a][0];
}
// distance(a, b) = dep[a] + dep[b] - 2 * dep[lca(a, b)]
// k-th ancestor: jump bits of k through up[][]
```

```
// tree diameter (unweighted): BFS twice
pair<int, int> farthest(int s, vector<vector<int>>& g) {
    vector<int> d(g.size(), -1);
    queue<int> q; q.push(s); d[s] = 0;
    int best = s;
    while (!q.empty()) {
        int u = q.front(); q.pop();
        if (d[u] > d[best]) best = u;
        for (int v : g[u])
            if (d[v] < 0) { d[v] = d[u] + 1; q.push(v); }
    }
    return {best, d[best]};
}
int diameter(vector<vector<int>>& g) {
    int a = farthest(1, g).first;
    // edges on the longest path
    return farthest(a, g).second;
}
```

7.8 Max flow (Dinic) and bipartite matching

```
struct Dinic {
    struct E { int to; ll cap; };
    vector<E> e;
    vector<vector<int>> g;
    vector<int> lvl, it;
    Dinic(int n) : g(n + 1), lvl(n + 1), it(n + 1) {}
    // directed edge u -> v
    void add(int u, int v, ll c) {
```

```
        g[u].push_back(e.size()); e.push_back({v, c});
        g[v].push_back(e.size()); e.push_back({u, 0});
    }
    bool bfs(int s, int t) {
        fill(lvl.begin(), lvl.end(), -1);
        lvl[s] = 0;
        queue<int> q; q.push(s);
        while (!q.empty()) {
            int u = q.front(); q.pop();
            for (int id : g[u])
                if (e[id].cap > 0 && lvl[e[id].to] < 0) {
                    lvl[e[id].to] = lvl[u] + 1;
                    q.push(e[id].to);
                }
        }
        return lvl[t] >= 0;
    }
    ll dfs(int u, int t, ll f) {
        if (u == t) return f;
        for (int& i = it[u]; i < (int)g[u].size(); i++) {
            int id = g[u][i], v = e[id].to;
            if (e[id].cap > 0 && lvl[v] == lvl[u] + 1) {
                ll r = dfs(v, t, min(f, e[id].cap));
                if (r > 0) {
                    e[id].cap -= r; e[id ^ 1].cap += r;
                    return r;
                }
            }
        }
        return 0;
    }
    ll maxflow(int s, int t) {
        ll flow = 0;
        while (bfs(s, t)) {
            fill(it.begin(), it.end(), 0);
            while (ll f = dfs(s, t, INF)) flow += f;
        }
        return flow;
    }
};
// undirected edge: add(u, v, c) and add(v, u, c)
// bipartite matching via flow: source -> left (cap 1),
// left -> right (cap 1), right -> sink (cap 1)
```

```
// Bipartite matching (Kuhn), O(V * E)
// gl[left] = right nodes
vector<vector<int>> gl;
// mt[right] = matched left
vector<int> mt;
vector<bool> seen;
bool kuhn(int u) {
    for (int v : gl[u]) {
        if (seen[v]) continue;
        seen[v] = true;
        if (mt[v] < 0 || kuhn(mt[v])) {
            mt[v] = u;
            return true;
        }
    }
    return false;
}
int matching(int L, int R) {
    mt.assign(R + 1, -1);
    int res = 0;
    for (int u = 1; u <= L; u++) {
        seen.assign(R + 1, false);
        res += kuhn(u);
    }
    return res;
}
```

7.9 Bipartite check

```
bool isBipartite(int n, vector<vector<int>>& g) {
    vector<int> c(n + 1, -1);
    for (int s = 1; s <= n; s++) {
        if (c[s] >= 0) continue;
        c[s] = 0;
        queue<int> q; q.push(s);
        while (!q.empty()) {
            int u = q.front(); q.pop();
            for (int v : g[u]) {
                if (c[v] < 0) {
                    c[v] = c[u] ^ 1;
                    q.push(v);
                }
                else if (c[v] == c[u]) return false;
            }
        }
    }
    return true;
}
```

8. Dynamic Programming

8.1 Max subarray sum and max product subarray

```
ll maxSubarraySum(vector<ll>& v) {       // Kadane
    ll cur = 0, ans = LLONG_MIN;
    for (ll x : v) {
        // start fresh or extend
        cur = max(x, cur + x);
        ans = max(ans, cur);
    }
    return ans;
}
ll maxProductSubarray(vector<ll>& v) {   // watch overflow
    ll mx = v[0], mn = v[0], ans = v[0];
    for (size_t i = 1; i < v.size(); i++) {
        ll x = v[i];
        if (x < 0) swap(mx, mn);
        mx = max(x, mx * x);
        mn = min(x, mn * x);
        ans = max(ans, mx);
    }
    return ans;
}
```

8.2 Longest increasing subsequence

```
// strictly increasing, O(n log n)
int lis(vector<int>& a) {
    // t[i] = smallest tail of length i+1
    vector<int> t;
    for (int x : a) {
        auto it = lower_bound(t.begin(), t.end(), x);
        if (it == t.end()) t.push_back(x);
        else *it = x;
    }
    // non-decreasing: use upper_bound
    return t.size();
}
```

8.3 LCS and edit distance

```
int lcs(const string& a, const string& b) {
    int n = a.size(), m = b.size();
    vector<vector<int>> dp(n + 1, vector<int>(m + 1, 0));
    for (int i = 1; i <= n; i++)
        for (int j = 1; j <= m; j++)
            dp[i][j] = a[i - 1] == b[j - 1]
                ? dp[i - 1][j - 1] + 1
                : max(dp[i - 1][j], dp[i][j - 1]);
    return dp[n][m];
}
int editDistance(const string& a, const string& b) {
    int n = a.size(), m = b.size();
    vector<vector<int>> dp(n + 1, vector<int>(m + 1, 0));
    for (int i = 0; i <= n; i++) dp[i][0] = i;
    for (int j = 0; j <= m; j++) dp[0][j] = j;
    for (int i = 1; i <= n; i++)
        for (int j = 1; j <= m; j++)
            dp[i][j] = min({dp[i - 1][j] + 1,
                dp[i][j - 1] + 1,
                dp[i - 1][j - 1] + (a[i - 1] != b[j - 1])});
    return dp[n][m];
}
```

8.4 Knapsack and coin change

```
ll knap01(vector<int>& wt, vector<ll>& val, int W) {
    vector<ll> dp(W + 1, 0);
    for (size_t i = 0; i < wt.size(); i++)
        // descending: 0/1
        for (int w = W; w >= wt[i]; w--)
            dp[w] = max(dp[w], dp[w - wt[i]] + val[i]);
    return dp[W];
}
// unbounded knapsack: same loop with w ascending
int minCoins(vector<int>& c, int T) {
    vector<int> dp(T + 1, INT_MAX);
    dp[0] = 0;
    for (int s = 1; s <= T; s++)
        for (int x : c)
            if (x <= s && dp[s - x] != INT_MAX)
                dp[s] = min(dp[s], dp[s - x] + 1);
    return dp[T] == INT_MAX ? -1 : dp[T];
}
// combinations, not orders
ll coinWays(vector<int>& c, int T) {
    vector<ll> dp(T + 1, 0);
    dp[0] = 1;
    for (int x : c)                    // coins outer
        for (int s = x; s <= T; s++) dp[s] += dp[s - x];
    // swap loops for ordered ways
    return dp[T];
}
```

8.5 Bitmask DP (travelling salesman)

```
ll tsp(vector<vector<ll>>& d) {        // 0-indexed, n <= 20
    int n = d.size();
    vector<vector<ll>> dp(1 << n, vector<ll>(n, INF));
    dp[1][0] = 0;                      // start at city 0
    for (int m = 1; m < (1 << n); m++)
        for (int i = 0; i < n; i++) {
            if (dp[m][i] == INF || !(m >> i & 1)) continue;
            for (int j = 0; j < n; j++) {
                if (m >> j & 1) continue;
                int nm = m | 1 << j;
                dp[nm][j] = min(dp[nm][j],
                                dp[m][i] + d[i][j]);
            }
        }
    ll ans = INF;
    for (int i = 1; i < n; i++)
        ans = min(ans, dp[(1 << n) - 1][i] + d[i][0]);
    return ans;
}
```

8.6 Subset sums (SOS DP) and submask enumeration

```
const int B = 4;
vector<ll> f(1 << B, 1);
// f[mask] = sum of f[sub] over all submasks of mask
for (int i = 0; i < B; i++)
    for (int m = 0; m < (1 << B); m++)
        if (m >> i & 1) f[m] += f[m ^ (1 << i)];
// enumerate all submasks of m (including 0), total 3^n
int mm = 11;
for (int s = mm; ; s = (s - 1) & mm) {
    // use s
    if (s == 0) break;
}
```

8.7 Digit DP skeleton

```
// upper bound N as a string
string S;
// pos, tight, digit sum so far
ll memo[20][2][200];
bool check(int digitSum) { return digitSum % 3 == 0; }
ll go(int pos, bool tight, int sum) {
    if (pos == (int)S.size()) return check(sum);
    ll& r = memo[pos][tight][sum];
    if (r != -1) return r;
    r = 0;
    int lim = tight ? S[pos] - '0' : 9;
    for (int d = 0; d <= lim; d++)
        r += go(pos + 1, tight && d == lim, sum + d);
    return r;
}
// count x in [0, N]: S = to_string(N); memo filled with -1
// go(0, true, 0). For [L, R]: f(R) - f(L - 1)
```

8.8 Prefix sums and difference arrays

```
// 1D, 1-indexed: pref[i] = v[0] + ... + v[i-1]
vector<ll> pref(n + 1, 0);
for (int i = 0; i < n; i++) pref[i + 1] = pref[i] + v[i];
// sum v[l..r] (0-indexed) = pref[r + 1] - pref[l]
// 2D prefix sum over a (n x m)
vector<vector<int>> a2(n, vector<int>(m, 1));
vector<vector<long long>> p2(n + 1,
                             vector<long long>(m + 1));
for (int i = 1; i <= n; i++) {
    for (int j = 1; j <= m; j++) {
        p2[i][j] = a2[i - 1][j - 1]
                 + p2[i - 1][j]
                 + p2[i][j - 1]
                 - p2[i - 1][j - 1];
    }
}
// sum of rows r1..r2, cols c1..c2 (1-indexed inclusive)
// = p2[r2][c2] - p2[r1-1][c2]
//   - p2[r2][c1-1] + p2[r1-1][c1-1]
// difference array: add val to [l, r] in O(1)
vector<ll> diff(n + 2, 0);
diff[l] += p; diff[r + 1] -= p;
for (int i = 1; i <= n; i++) diff[i] += diff[i - 1];
```

9. Data Structures

9.1 Fenwick tree (BIT)

```
// point update, prefix sum, 1-indexed
struct BIT {
    int n;
    vector<ll> t;
    BIT(int n) : n(n), t(n + 1, 0) {}
    void add(int i, ll v) {
        for (; i <= n; i += i & -i) t[i] += v;
    }
    ll sum(int i) {
        ll s = 0;
        for (; i > 0; i -= i & -i) s += t[i];
        return s;
    }
    ll sum(int l, int r) { return sum(r) - sum(l - 1); }
};
// count inversions: iterate left to right, add(a[i], 1),
// inversions += i - sum(a[i])   (values compressed to 1..n)
```

9.2 Segment tree (iterative, point update)

```
// range sum on [l, r), 0-indexed
struct Seg {
    int n;
    vector<ll> t;
    Seg(int n) : n(n), t(2 * n, 0) {}
    void upd(int i, ll v) {
        for (t[i += n] = v; i > 1; i >>= 1)
            t[i >> 1] = t[i] + t[i ^ 1];
    }
    ll query(int l, int r) {
        ll res = 0;
        for (l += n, r += n; l < r; l >>= 1, r >>= 1) {
            if (l & 1) res += t[l++];
            if (r & 1) res += t[--r];
        }
        return res;
    }
};
// min/max: use min, res = INF, t initialized to INF
```

9.3 Lazy segment tree (range add, range sum)

```
// 1-indexed, [l, r] inclusive
struct LazySeg {
    int n;
    vector<ll> s, lz;
    LazySeg(int n) : n(n), s(4 * n, 0), lz(4 * n, 0) {}
    void app(int x, int l, int r, ll v) {
        s[x] += v * (r - l + 1);
        lz[x] += v;
    }
    void push(int x, int l, int r) {
        if (!lz[x]) return;
        int m = (l + r) / 2;
        app(2 * x, l, m, lz[x]);
        app(2 * x + 1, m + 1, r, lz[x]);
        lz[x] = 0;
    }
    void add(int x, int l, int r, int ql, int qr, ll v) {
```

```
        if (qr < l || r < ql) return;
        if (ql <= l && r <= qr) { app(x, l, r, v); return; }
        push(x, l, r);
        int m = (l + r) / 2;
        add(2 * x, l, m, ql, qr, v);
        add(2 * x + 1, m + 1, r, ql, qr, v);
        s[x] = s[2 * x] + s[2 * x + 1];
    }
    ll query(int x, int l, int r, int ql, int qr) {
        if (qr < l || r < ql) return 0;
        if (ql <= l && r <= qr) return s[x];
        push(x, l, r);
        int m = (l + r) / 2;
        return query(2 * x, l, m, ql, qr)
             + query(2 * x + 1, m + 1, r, ql, qr);
    }
    void add(int l, int r, ll v) { add(1, 1, n, l, r, v); }
    ll query(int l, int r) { return query(1, 1, n, l, r); }
};
```

9.4 Sparse table (static range minimum)

```
struct Sparse {
    vector<vector<int>> t;
    Sparse(vector<int>& a) {
        int n = a.size(), K = __lg(n) + 1;
        t.assign(K, vector<int>(n));
        t[0] = a;
        for (int k = 1; k < K; k++)
            for (int i = 0; i + (1 << k) <= n; i++)
                t[k][i] = min(t[k - 1][i],
                              t[k - 1][i + (1 << (k - 1))]);
    }
    // min of a[l..r], O(1)
    int query(int l, int r) {
        int k = __lg(r - l + 1);
        return min(t[k][l], t[k][r - (1 << k) + 1]);
    }
};
```

9.5 Monotonic stack and deque

```
// next greater element index (or -1)
vector<int> nge(n, -1);
stack<int> stk;
for (int i = 0; i < n; i++) {
    while (!stk.empty() && v[stk.top()] < v[i]) {
        nge[stk.top()] = i;
        stk.pop();
    }
    stk.push(i);
}
// previous smaller: pop while top >= v[i]
```

```
vector<int> slidingMax(vector<int>& a, int k) {
    // indices, values decreasing
    deque<int> dq;
    vector<int> res;
    for (int i = 0; i < (int)a.size(); i++) {
        while (!dq.empty() && a[dq.back()] <= a[i])
            dq.pop_back();
        dq.push_back(i);
        if (dq.front() <= i - k) dq.pop_front();
        if (i >= k - 1) res.push_back(a[dq.front()]);
    }
    return res;
}
```

9.6 Two pointers and coordinate compression

```
// longest subarray with sum <= p (non-negative values)
int lft = 0, best = 0;
ll cur = 0;
for (int rgt = 0; rgt < n; rgt++) {
    cur += v[rgt];
    while (cur > p && lft <= rgt) cur -= v[lft++];
    best = max(best, rgt - lft + 1);
}
// pair with sum == target in a SORTED array
int i = 0, j = n - 1;
while (i < j) {
    int s2 = v[i] + v[j];
    if (s2 == target) { i++; j--; }
    else if (s2 < target) i++;
    else j--;
}
// coordinate compression
vector<int> xs = v;
sort(all(xs));
xs.erase(unique(all(xs)), xs.end());
// rank 0..k-1
int id = lower_bound(all(xs), x) - xs.begin();
```

9.7 Ordered set (policy based data structure)

```
#include <bits/stdc++.h>
#include <ext/pb_ds/assoc_container.hpp>
#include <ext/pb_ds/tree_policy.hpp>
using namespace std;
using namespace __gnu_pbds;
typedef tree<int, null_type, less<int>, rb_tree_tag,
             tree_order_statistics_node_update> ordered_set;
// ordered_set os; os.insert(x); os.erase(x);
// *os.find_by_order(k)  : k-th smallest, 0-indexed
// os.order_of_key(x)    : count of elements strictly < x
// duplicates: insert pair<int,int>(value, uniqueId)
```

10. Number Theory and Math

10.1 Modular arithmetic, fast power, nCr

```
ll power(ll a, ll b, ll mod = MOD) {
    ll ans = 1;
    a %= mod;
    while (b > 0) {
        if (b & 1) ans = ans * a % mod;
        a = a * a % mod;
        b >>= 1;
    }
    return ans;
}
// MOD must be prime
ll inv(ll a) { return power(a, MOD - 2); }
// add: (a + b) % MOD       multiply: a * b % MOD
// subtract: ((a - b) % MOD + MOD) % MOD
// divide: a * inv(b) % MOD
const int MX = 1000001;
ll fact[MX], ifact[MX];
void initFact() {
    fact[0] = 1;
    for (int i = 1; i < MX; i++)
        fact[i] = fact[i - 1] * i % MOD;
    ifact[MX - 1] = inv(fact[MX - 1]);
    for (int i = MX - 1; i > 0; i--)
        ifact[i - 1] = ifact[i] * i % MOD;
}
ll nCr(int n, int r) {
    if (r < 0 || r > n) return 0;
    return fact[n] * ifact[r] % MOD * ifact[n - r] % MOD;
}
```

10.2 GCD, extended GCD, modular inverse (any modulus)

```
ll gcdll(ll a, ll b) { return b ? gcdll(b, a % b) : a; }
// extgcd: finds x, y with a*x + b*y = gcd(a, b)
ll extgcd(ll a, ll b, ll& x, ll& y) {
    if (!b) { x = 1; y = 0; return a; }
    ll g = extgcd(b, a % b, y, x);
    y -= a / b * x;
    return g;
}
ll invMod(ll a, ll m) {              // needs gcd(a, m) == 1
    ll x, y;
    extgcd(a, m, x, y);
    return (x % m + m) % m;
}
// lcm(a, b) = a / gcd(a, b) * b
// a*x + b*y = c solvable iff gcd(a, b) divides c
```

10.3 Sieve of Eratosthenes and smallest prime factor

```
vector<int> sieve(int n) {           // primes up to n
    vector<bool> isPrime(n + 1, true);
    isPrime[0] = isPrime[1] = false;
    for (int i = 2; (ll)i * i <= n; i++) {
        if (!isPrime[i]) continue;
        for (int j = i * i; j <= n; j += i)
            isPrime[j] = false;
    }
    vector<int> primes;
    for (int i = 2; i <= n; i++)
        if (isPrime[i]) primes.push_back(i);
    return primes;
}
// smallest prime factor
vector<int> spf;
void buildSpf(int n) {
    spf.assign(n + 1, 0);
    for (int i = 2; i <= n; i++)
        if (!spf[i])
            for (int j = i; j <= n; j += i)
                if (!spf[j]) spf[j] = i;
}
// factorize x fast: while (x > 1) { p = spf[x]; x /= p; }
```

10.4 Factorization, divisors, Euler phi

```
vector<pair<long long, int>> factorize(long long n) {
    vector<pair<long long, int>> factors;
    for (long long p = 2; p * p <= n; p++) {
        if (n % p != 0) continue;
        int cnt = 0;
        while (n % p == 0) {
            n /= p;
            cnt++;
        }
        factors.push_back({p, cnt});
    }
    if (n > 1) factors.push_back({n, 1});
    return factors;                  // {prime, exponent}
}
vector<ll> divisors(ll n) {
    vector<ll> d;
    for (ll i = 1; i * i <= n; i++)
        if (n % i == 0) {
            d.push_back(i);
            if (i != n / i) d.push_back(n / i);
        }
    sort(d.begin(), d.end());
    return d;
}
// count of k <= n coprime to n
ll phi(ll n) {
    ll r = n;
    for (ll p = 2; p * p <= n; p++)
        if (n % p == 0) {
            while (n % p == 0) n /= p;
            r -= r / p;
        }
    if (n > 1) r -= r / n;
    return r;
}
```

10.5 Matrix exponentiation

```
typedef vector<vector<ll>> Mat;
Mat mul(const Mat& A, const Mat& B) {
    int n = A.size();
    Mat C(n, vector<ll>(n, 0));
    for (int i = 0; i < n; i++)
        for (int k = 0; k < n; k++) {
            if (!A[i][k]) continue;
            for (int j = 0; j < n; j++)
                C[i][j] = (C[i][j] + A[i][k] * B[k][j])
                          % MOD;
        }
    return C;
}
Mat mpow(Mat A, ll e) {
    int n = A.size();
    Mat R(n, vector<ll>(n, 0));
    for (int i = 0; i < n; i++) R[i][i] = 1;
    while (e) {
        if (e & 1) R = mul(R, A);
        A = mul(A, A);
        e >>= 1;
    }
    return R;
}
// Fibonacci: mpow({{1, 1}, {1, 0}}, n)[0][1] = F(n)
```

10.6 Miller-Rabin primality (64-bit)

```
ull mulmod(ull a, ull b, ull m) {
    return (__uint128_t)a * b % m;
}
ull powmod(ull a, ull e, ull m) {
    ull r = 1;
    a %= m;
    while (e) {
        if (e & 1) r = mulmod(r, a, m);
        a = mulmod(a, a, m);
        e >>= 1;
    }
    return r;
}
const ull BASES[] = {2, 3, 5, 7, 11, 13, 17, 19, 23, 29,
                     31, 37};
bool isPrimeMR(ull n) {
    if (n < 2) return false;
    for (ull p : BASES)
        if (n % p == 0) return n == p;
    ull d = n - 1;
    int r = 0;
    while (d % 2 == 0) { d /= 2; r++; }
    for (ull a : BASES) {
        ull x = powmod(a, d, n);
        if (x == 1 || x == n - 1) continue;
        bool composite = true;
        for (int i = 1; i < r; i++) {
            x = mulmod(x, x, n);
            if (x == n - 1) { composite = false; break; }
        }
        if (composite) return false;
    }
    return true;
}
```

10.7 Formulas

Quantity	Formula
sum 1..n	n(n+1)/2
sum of squares	n(n+1)(2n+1)/6
sum of cubes	(n(n+1)/2)^2
arithmetic series	n(a1+an)/2
geometric series	a(r^n - 1)/(r - 1)
nCr	n! / (r!(n-r)!), nCr = nC(n-r)
Pascal	C(n,r) = C(n-1,r-1) + C(n-1,r)
stars and bars (x1+..+xk = n, xi >= 0)	C(n+k-1, k-1)
Catalan	C(2n,n)/(n+1): 1 1 2 5 14 42 132 429
derangements	D(n) = (n-1)(D(n-1)+D(n-2))
Fermat	a^(p-1) = 1 mod p (p prime, p not dividing a)
Euler	a^phi(m) = 1 mod m (gcd(a,m)=1)
a^b mod m, huge b	reduce b mod phi(m) when gcd(a,m)=1
number of divisors	product of (e_i + 1)

Quantity	Formula
sum of divisors	product of (p^(e+1) - 1)/(p - 1)
ceil(a/b) positive	(a + b - 1) / b
gcd(a,b)	gcd(b, a mod b), gcd(a,b)*lcm(a,b) = a*b
inclusion-exclusion	add odd-size subset intersections, subtract even-size ones
expected value	sum of value * probability (linearity helps)

11. Searching Tricks

11.1 Binary search on the answer

```
// smallest x in [lo, hi] with ok(x) true (F F T T T)
auto ok = [&](ll mid) { return mid * mid >= p; };
ll lo = 0, hi = 2e9;                  // ok(hi) must be true
while (lo < hi) {
    ll mid = lo + (hi - lo) / 2;
    if (ok(mid)) hi = mid;
    else lo = mid + 1;
}
// answer is lo
// largest x with ok2(x) true (pattern T T T F F F)
auto ok2 = [&](ll mid) { return mid * mid <= p; };
ll lo2 = 0, hi2 = 2e9;
while (lo2 < hi2) {
    ll mid = lo2 + (hi2 - lo2 + 1) / 2;   // round UP
    if (ok2(mid)) lo2 = mid;
    else hi2 = mid - 1;
}
// answer is lo2
// doubles: loop 100 times with mid = (lo + hi) / 2
```

11.2 Ternary search (unimodal function)

```
auto f = [&](double z) { return (z - 3) * (z - 3); };
// minimum of f on [lo, hi]
double lo = 0, hi = 1e9;
for (int it = 0; it < 100; it++) {
    double m1 = lo + (hi - lo) / 3;
    double m2 = hi - (hi - lo) / 3;
    if (f(m1) < f(m2)) hi = m2;
    else lo = m1;
}
// answer f(lo). Integer version: loop while hi - lo > 2
```

12. Geometry

12.1 Points, cross product, orientation

```
struct P {
    ll x, y;
    P operator-(const P& o) const {
        return {x - o.x, y - o.y};
    }
    P operator+(const P& o) const {
        return {x + o.x, y + o.y};
    }
    bool operator<(const P& o) const {
        return tie(x, y) < tie(o.x, o.y);
    }
    bool operator==(const P& o) const {
        return x == o.x && y == o.y;
    }
};
ll cross(P a, P b) { return a.x * b.y - a.y * b.x; }
ll dot(P a, P b) { return a.x * b.x + a.y * b.y; }
ll cross3(P o, P a, P b) { return cross(a - o, b - o); }
// cross3 > 0: left turn (CCW), < 0: right, 0: collinear
// squared distance: dot(a - b, a - b), no sqrt needed
// angle: atan2(dy, dx) in (-pi, pi], pi = acos(-1.0)
// point p on segment ab: cross3(a, b, p) == 0 and
//   min(a.x,b.x) <= p.x <= max(a.x,b.x), same for y
```

12.2 Convex hull and polygon area

```
// Andrew monotone chain
vector<P> convexHull(vector<P> p) {
    sort(p.begin(), p.end());
    p.erase(unique(p.begin(), p.end()), p.end());
    int n = p.size();
    if (n < 3) return p;
    vector<P> h(2 * n);
    int k = 0;
    for (int i = 0; i < n; i++) {
        while (k >= 2 &&
               cross3(h[k - 2], h[k - 1], p[i]) <= 0) k--;
        h[k++] = p[i];
    }
    for (int i = n - 2, t = k + 1; i >= 0; i--) {
        while (k >= t &&
               cross3(h[k - 2], h[k - 1], p[i]) <= 0) k--;
        h[k++] = p[i];
    }
    h.resize(k - 1);
    // counter clockwise, no collinear
    return h;
}
ll area2(const vector<P>& p) {             // twice the area
    ll s = 0;
    int n = p.size();
    for (int i = 0; i < n; i++)
        s += cross(p[i], p[(i + 1) % n]);
    return llabs(s);
}
```

13. Bit Manipulation

13.1 Common tricks

Goal	Expression
test bit k	`(x >> k) & 1`
set bit k	`x | (1 << k)` (use 1LL for k >= 31)
clear bit k	`x & ~(1 << k)`
toggle bit k	`x ^ (1 << k)`
lowest set bit	`x & -x`
clear lowest set bit	`x & (x - 1)`
power of two?	`x > 0 && (x & (x - 1)) == 0`
all ones of length k	`(1 << k) - 1`
multiply, divide by 2^k	`x << k`, `x >> k`
swap without temp	`a ^= b; b ^= a; a ^= b;`
x xor x, x xor 0	0 and x (find the single unpaired number)
a + b	`(a ^ b) + 2 * (a & b)`
popcount, ctz, clz	see 4.9

13.2 Enumerating masks

```
// all subsets
for (int mask = 0; mask < (1 << n); mask++) { }
for (int mask = 0; mask < (1 << n); mask++)
    // size k only
    if (__builtin_popcount(mask) == k) { }
int full = (1 << n) - 1;
// complement
int comp = full ^ x;
// iterate set bits of m
int m3 = 44;
for (int t2 = m3; t2; t2 &= t2 - 1) {
    // index of bit
    int bit = __builtin_ctz(t2);
}
// submasks of a mask: see 8.6
```

14. Pitfalls and Constants

14.1 Contest checklist

- Overflow: use `long long` when values or products reach 1e9 or more. Cast before multiplying (`1LL * a * b`).
- `v.size() - 1` is unsigned: when empty it wraps to a huge number. Use `(int)v.size() - 1`.
- Reset global arrays and containers between test cases.
- `endl` flushes every time and is slow. Use `"\n"`.
- Negative modulo: `(a % m + m) % m`. In Python `%` is already non-negative.
- `map[key]` inserts the key. Use `count` or `find` to test existence.

- `set.erase(it)` invalidates only that iterator. In a loop use `it = st.erase(it)`.
- `multiset.erase(x)` removes all copies, `erase(find(x))` removes one.
- `pow()` returns a double: do not use it for exact integers. Write your own.
- Floating point: compare with eps (1e-9), print with `setprecision`.
- Integer division rounds toward zero in C++ (floors in Python).
- `lower_bound` on `set` with `std::lower_bound` is slow. Use the member `st.lower_bound(x)`.
- Sorting with <= in a comparator can crash. Use strict <.
- Off by one: check < vs <=, 0-indexed vs 1-indexed.
- Recursion depth of about 1e5 may overflow the stack: go iterative.
- Uninitialized variables: always initialize (`int cnt = 0;`).
- Read input exactly as described (n then n numbers). Test on the samples and on `n = 1`.
- Dijkstra needs non-negative weights. Use Bellman-Ford for negative edges.
- Use `INF` that survives addition: `1e18` for ll, `1e9` for int sums (not `INT_MAX`).

14.2 Constants and tables

Item	Value
primes for mod	1000000007, 998244353, 1000000009
2^10, 2^20, 2^30, 2^31	1024, 1048576, 1073741824, 2147483648
2^32, 2^40, 2^62, 2^63	4294967296, 1.1e12, 4.6e18, 9.2e18
10! , 12!, 13!	3628800, 479001600, 6227020800 (13! > INT_MAX)
15!, 20!	1.3e12, 2.4e18 (21! overflows ll)
primes below 1e3, 1e6, 1e9	168, 78498, 50847534
max divisors (n <= 1e6, 1e9, 1e18)	240, 1344, 103680
Fibonacci F(40), F(90)	102334155, 2.88e18
ASCII: '0', 'A', 'a'	48, 65, 97 (a - A = 32)
pi, e	3.14159265358979, 2.71828182845905
log2(1e9), log2(1e18)	about 30, about 60